核心文献深读：TAF 与"未说完就开始翻译"

> 配套： 研究方案_LLM流式同传并行化.md 、 文献详解_同传与LLM推测.md
> 这份文档不是文献索引，而是对 **TAF 及最直接相关的 12 篇工作** 做逐篇拆解：问题假设、方法细节、实验、为什么有效、局限、与 TAF/PFBA 的差异、可执行借鉴。

0. 选文标准与阅读地图

主线问题只有五个：

1. **何时写**：READ/WRITE 的时机；
2. **写什么**：提交哪些目标词；
3. **未来从哪来**：源未来如何获得（预测、蒸馏、目标侧 lookahead、上一轮输出）；
4. **如何保证一致**：投票、LCP、RALCP、验证、回退；
5. **并行预算**：B 条分支如何在 Δ 内跑完（$\rho(B) < 1$）。

按这五个问题，本文选 12 篇：

组	文献	回答的问题
A. 同构方案	**TAF**、Speculative Beam Search、SSBD	未来从哪来、如何提交
B. 提交/一致性	STACL、Liu 2020、Hoang stable prefix、ALTA/RALCP	写什么、如何一致
C. 未来信息/训练	Future-Guided、Glancing Future、Anticipation-Free、NAST	未来信息能否训练进模型
D. LLM 机制	Speculative Decoding、Self-Consistency、Mask-Predict、FLARE、Quiet-STaR、Simul-LLM、TransLLaMa	可直接搬用的机制

1. TAF 深读

Anticipating Future with Large Language Model for Simultaneous Machine Translation
Siqi Ouyang, Oleksii Hrinchuk, Zhehuai Chen, Vitaly Lavrukhin, Jagadeesh Balam, Lei Li, Boris Ginsburg
arXiv:2410.22499（v2, 2025）链接｜代码：https://github.com/owaski/TAF

1.1 问题形式化

设源 $x = (x_1, \ldots, x_L)$，目标 $y = (y_1, \ldots, y_M)$，g_i 表示生成 y_i 时已读的源词数。策略

$$\pi\left(x_{1:j}, y_{1:i}\right) \in [0, 1]$$

表示在已读 $x_{1:j}$、已生成 $y_{1:i}$ 时**继续写** y_{i+1} 的概率；$1-\pi$ 表示**继续读**。TAF 的核心不是改 MT 模型，而是改策略：用 LLM 预测未来源词，让 MT 模型"提前看到"可能的后续。

Oracle 形式：如果知道真实的源语言模型 P^*_{LM}，理想的同传翻译应该是对未来源取期望：

$$P^*_{MT}\left(y_{i+1} \mid x_{\le j}, y_{\le i}\right) = \mathbb{E}_{x_{j+1:} \sim P^*_{LM}(\cdot \mid x_{\le j})}\left[P_{MT}\left(y_{i+1} \mid x_{\le j}, x_{j+1:}, y_{\le i}\right)\right]$$

这个式子是整篇论文的理论支点：**同传的不确定性可以被"对未来取期望"吸收**。TAF 用预训练 LLM 近似 P^*_{LM}，用采样近似期望。

1.2 算法逐步拆解

Step 1：采样未来

$$\hat{x}^{(b)}_{j+1:j+l} \sim P_{LM}\left(\cdot \mid x_{\le j}\right), \qquad b = 1, \ldots, n$$

默认 $n = 10$，$l = 10$，top-$k = 10$。论文指出：更远的未来对当前词影响小，所以 l 不需要无限长。

Step 2：逐分支翻译

$$P^b_{MT} = P_{MT}\left(\cdot \mid x_{\le j}, \hat{x}^{(b)}_{j+1:j+l}, y_{\le i}\right)$$

每个分支是"已读前缀 + 一个想象出来的未来"，送进同一个 MT 模型。

Step 3：多数投票

$$y_{i+1} = \mathrm{Majority}\left(\arg\max P^1_{MT}, \ldots, \arg\max P^n_{MT}\right)$$

Step 4：阈值策略

$$\pi\left(x_{1:j}, y_{1:i}\right) = \frac{1}{n}\,\mathrm{Count}\left(y_{i+1}\right)$$

若 $\pi \ge \tau$ 就 WRITE，否则 READ。τ 控制质量-延迟折中。

兼容现有策略：TAF 可以套在 wait-k、LA-2、RALCP 上。论文强调：套在 RALCP 上时，$n\times$ beam width 与 RALCP 的 beam width 对齐，保证公平比较。

1.3 实验设计与结果

数据：

- De-En：WMT15 训练（4.5M 句），newstest2013 验证，newstest2015 测试；
- En-Zh / Zh-En：WMT20（约 40M 句），newstest2019 验证，newstest2020 测试；
- En-Ja：WMT23（约 29M 句），newstest2020 验证，newstest2023 测试。

模型：

- MT：Transformer-Big（0.2B）；另测 MADLAD-400-3B-MT、TowerInstruct-7B；
- LLM：Llama 2 7B base；另测 MicroLlama-300M、Mistral-7B、TowerInstruct-7B。

指标：BLEU、COMET、LAAL（按词/字符，而不是 BPE）。

基线：SM2（SOTA 自适应策略）、Wait-K-Stride-N、LA-2、RALCP。

主要结果：

- RALCP + TAF 在四个方向上最好；
- En-Zh 在 3 词延迟下最多 **+5 BLEU**；
- 在约 2 词延迟下，TAF 对 Wait-K-Stride-N、LA-2、RALCP 均有至少 **+6 BLEU**；
- 跨 MT/LLM 组合普遍有效：Transformer-Big / MADLAD-400-3B 上 3 词延迟 +5 BLEU 以上；TowerInstruct-7B 上 +8 BLEU 以上；
- MicroLlama-300M 会掉不到 2 BLEU，但仍比 base RALCP 高 4 BLEU。

超参数敏感性：

- l 增大后收益在 $l \geq 6$ 饱和；
- n 增大后收益在 $n \geq 4$ 饱和；
- top-k 增大后收益在 $k \geq 8$ 饱和；
- 默认 $n = 10, l = 10, k = 10$ 偏保守，可以进一步降算力。

计算成本（论文 Table 1）：

MT 模型	规模	无 TAF	有 TAF
Transformer-Big	0.2B	72 ms	**1067 ms**
MADLAD-400	3B	383 ms	**1339 ms**
TowerInstruct	7B	905 ms	**1506 ms**

这组数字非常关键：**TAF 的算法收益是用 15×（小模型）到 1.7×（大模型）的单步墙钟时间换来的**。论文自己把"额外计算成本"列为主要局限。

人工分析（100 条 En-Zh，$\tau = 0.6$）：

模式	频率	ΔCOMET	ΔLAAL
正确预测 → 降延迟	44%	+10.61	−0.45
预测不对但提供额外上下文 → 降延迟	53%	+5.42	−1.05
额外上下文 → 减少幻觉	39%	+24.44	+0.29
LLM 偏见 → 引入幻觉	28%	−6.99	−0.31
总体	100%	+6.78	−0.25

注意频率可以重叠（一条实例可能同时属于多种模式）。这张表说明：**TAF 不是单纯"猜对就赚"；它有时靠"猜错但提供上下文"也能改善策略，但也会因 LLM 偏见引入新的幻觉。**

更长上下文：把前几句拼到 LLM 的 context 里（但不送给 MT），En-Zh 上再降 0.5–1 词延迟，质量不变。

1.4 为什么有效

1. **实体/日期/固定搭配高度可预测**：如"India and France ... 1998"，LLM 能猜出年份，MT 提前翻。
2. **即使猜错，未来采样也提供了"可能的世界"**：MT 模型看到多个可能续写后，对当前前缀的翻译更自信或更保守，从而改善 READ/WRITE 决策。
3. **多数投票过滤分歧**：如果 n 个未来给出的目标词不一致，说明当前信息不足，应该继续读；如果一致，说明可以写。

1.5 关键假设与隐含风险

假设	风险
预训练 LLM 近似真实源分布 P^*_{LM}	领域漂移、专有名词、低资源语言会失准
多数投票 = 一致性	如果所有分支都因同一偏见而错，投票会"自信地错"
固定 n, l 足够	不同句子/位置的不确定性差异很大；固定预算浪费或不足
采样未来足以覆盖真实未来	没有覆盖率保证；真实未来可能不在 n 个分支里
MT 模型能正确处理"前缀 + 想象未来"	想象未来可能污染 MT 的翻译，引入幻觉
额外算力可接受	0.2B MT 上单步 72→1067 ms，实时性被破坏

论文自己的局限：聚合函数只试了多数投票；只做 X-En/En-X；只做文本；语音需要级联 ASR，连续音频未来很难预测。

1.6 TAF 与 PFBA 的逐维对比

维度	TAF	PFBA（建议）
未来来源	LLM 采样 n 个	同 + ASR n-best/lattice
分支数	固定 n	自适应 B，受 $\rho(B) < 1$ 约束
提交规则	多数投票 + 阈值 τ	LCP/RALCP + 置信度
到达验证	**无**	真实未来到达后验证、保留/回退
安全性	可能被 LLM 偏见带偏	一致前缀 + 验证 + 回退
并行	未分析	分支批处理、共享前缀 KV、tree attention
实时预算	未分析	$B_{\max}$、Δ、GPU 利用率
测量	BLEU/COMET/LAAL	+ coverage、acceptance、oracle gap
语言	X-En / En-X	+ X-X、中文
模态	文本	文本 + ASR n-best

结论：TAF 已经把"采样未来 + 翻译 + 投票"做完了；PFBA 的空间在**系统层**（并行、预算、验证、安全、测量），而不是算法层的"未来分支"本身。

1.7 TAF 留下的五个可做问题

1. **分支并行**：n 个分支如何共享前缀 KV、批量解码、树形 attention；墙钟能降多少？
2. **实时预算**：给定 Δ，n 最大能取多少？$\rho(B) < 1$ 的边界在哪里？
3. **验证与回退**：真实未来到达后，如何判断哪个分支可信？如何回退而不闪烁？
4. **安全提交**：多数投票在"没有分支覆盖真实未来"时是否安全？如何用置信度/一致性控制 hallucination？
5. **可预测性测量**：oracle（真实未来）与 TAF 的差距有多大？哪些语言对/片段可预测？

1.8 复现 TAF 的最小实验

- 模型：Llama 2 7B base + Transformer-Big；
- 数据：WMT15 De-En（4.5M 训练，newstest2015 测试）；
- 设置：$n = 10$，$l = 10$，top-$k = 10$，$\tau \in [0.5, 1.0]$；
- 指标：BLEU、COMET、LAAL、单步墙钟；
- 先复现 RALCP 与 RALCP+TAF 的 Pareto，再记录覆盖率与单步延迟；
- 然后加：共享前缀 KV、分支批处理、真实未来验证、$B_{\max}$ 扫描。

2. 同构/近邻方案

2.1 Speculative Beam Search（目标侧 lookahead）

Zheng, Ma, Zheng, Huang, EMNLP 2019. 链接

问题：wait-k 每读一个源词只提交一个目标词，beam search 似乎无法使用；greedy 又容易受 label bias 影响。

方法：在当前步不只选最高分词，而是**向目标侧"幻觉" w 步未来**，用第 $w+1$ 步的排序反过来决定第 1 步（当前步）的输出。它对 wait-k 和自适应策略都适用；末尾源句结束后再切回常规 beam search。

为什么有效：它隐式利用了目标语言模型——一个当前词如果导致后续目标序列概率更高，就更可能是对的。这缓解了神经生成中的 label bias。

结果：多个语言对上大幅优于此前方法；在保持 simultaneity 的同时提升质量。

与 TAF/PFBA 的关系：

- 相同点：都是"向前看再决定现在"；
- 不同点：SBS 猜的是**目标未来**，没有引入新的源信息；TAF/PFBA 猜的是**源未来**，直接补足信息缺口；
- SBS 不解决词序等待，只改善当前词的决策。

可借鉴：lookahead window、用未来排序回推当前决策、与 wait-k/自适应策略兼容。

局限：目标侧幻觉可能放大错误；不增加源信息；没有并行预算分析。

2.2 SSBD：自推测加速 re-translation

Zeng, Deng, Shu, Wang, arXiv:2509.21740, 2025. 链接

问题：re-translation 质量好，但每来一个新源词就重翻整个目标前缀，LLM 上非常慢。

方法：

1. 把上一轮输出 Y^{T-1} 当作当前输入 X^T 的 speculative draft；
2. 一次前向并行验证，直到第一个不匹配位置 d；
3. 从 d 开始继续自回归解码；
4. **biased verification**：$P'(y_i) = (1-\beta)P(y_i) + \beta\,\delta(y_i, Y_i^{T-1})$，软性偏向 draft，提高接受率；
5. **display-only mask-k**：只在显示层隐藏最后 k 个不稳定 token，内部仍保留为 draft，兼顾稳定与复用。

结果：Flores/ACL 60/60 上 1.3–1.7× 加速，COMET 相当；NE（Normalized Erasure）闪烁指标下降；En-Ja 因词序差异大，接受率低于 En-De/En-Zh。

与 TAF/PFBA 的关系：

- SSBD 是**目标侧 draft**，TAF 是**源侧 future**；
- 两者都用"验证"来决定接受/回退；SSBD 的验证对象是"新源下的目标一致性"，PFBA 的验证对象是"真实未来是否匹配分支"；
- display-only mask-k 可以直接借来控制 PFBA 的 flicker。

可借鉴：draft 复用、first-divergence resume、biased verification、display-only masking、NE 指标。

局限：仍是 re-translation 范式；不预测源未来；β 太大会压 COMET。

2.3 小结："向前看"的路线

路线	看什么	信息来自	代表
源侧未来	源句可能怎么继续	LLM/ASR n-best	TAF、PFBA
目标侧 lookahead	目标句可能怎么继续	目标 LM	Speculative Beam Search
目标侧 draft	上一轮译文	时间一致性	SSBD

三者可以叠加：源侧分支决定"翻什么"，目标侧 lookahead/draft 决定"怎么快速写"。

3. 提交与一致性

3.1 STACL：wait-k 与 prefix-to-prefix

Ma et al., ACL 2019. 链接

方法：训练时枚举源前缀与目标前缀对，让模型见到 $(x_{\le i}, y_{\le j})$ 的各种组合；推理时 wait-k：先读 k 个源词，之后读 1 写 1，满足

$$i(t) - j(t) = k$$

为什么有效：模型在训练中见过大量"源不完整、目标也不完整"的配对，学会了在信息不足时"提前"输出（implicit anticipation）。例如 SOV→SVO 时，它学会在动词还没出现时先写英语动词。

结果：低延迟下质量可接受；wait-k 简单、强、可复现。

局限：

- k 固定，不区分可预测/不可预测片段；
- 强制 anticipation，可能幻觉；
- 没有显式未来分布，无法知道"猜得对不对"。

与 TAF/PFBA 的关系：wait-k 是"等"的基准；TAF 是"猜"的基准。两者是同一信息缺口下的两端。

可借鉴：prefix-to-prefix 训练、AL 指标、wait-k 基线。

3.2 Liu et al. 2020：LCP 提交与单向 encoder

Liu, Spanakis, Niehues, arXiv:2005.11185. 链接

方法：

1. 固定大小 chunk 到达；
2. 每次解码一个完整假设；
3. 只提交相邻假设的 LCP：

$$c_t = \mathrm{LCP}\left(h_t, h_{t-1}\right)$$

4. 用**单向 encoder** 屏蔽未来，避免新 chunk 到来时重算历史隐状态；
5. 对部分输入做适配训练。

结果：How2 上延迟降 83% 到 0.8 s，WER 仅 +1%；英葡语音翻译 0.7 s，BLEU −2.4。

为什么有效：LCP 是"稳定性"的代理——如果两次解码都同意的前缀，大概率不会因新输入而改变；单向 encoder 让增量计算成为可能。

与 TAF/PFBA 的关系：LCP 提交是 PFBA 一致前缀的祖先；TAF 的多数投票是 LCP 的"多未来版"。

可借鉴：LCP、单向 encoder、延迟指标。

局限：chunk 大小固定；不预测未来；LCP 只保证"两次一致"，不保证"正确"。

3.3 Hoang et al.：Stable-Prefix 训练

Hoang, Axelrod, Post, WMT 2026. 链接

问题：wait-k/target-suffix deletion 的提交目标太粗；训练目标应该匹配"前缀到底能稳定翻出多少"。

方法：

1. 对每个源前缀 $s_{1:i}$，用翻译系统得到 $t(s_{1:i})$；
2. 与全句翻译 $f = t(s)$ 求 LCP：

$$m_i = |\mathrm{LCP}\left(t(s_{1:i}), f\right)|, \qquad p'_i = f_{1:m_i}$$

3. 用 $(s_{1:i}, p'_i)$ 作为训练对；
4. 微调 Qwen3-8B，得到 prefix 模型（首段）与 continuation 模型（续写）；
5. 推理时用置信度阈值控制提交，flicker-free。

结果：en-de/ja/zh 上改善 latency-quality Pareto；FLEURS、WMT24++、CoVoST2 均有效。

为什么有效：stable prefix 是"在只看到前缀时，能安全提交的最大目标前缀"；训练目标与推理时的提交行为一致，减少了 forced prediction。

与 TAF/PFBA 的关系：TAF 解决"未来从哪来"，Hoang 解决"提交什么"；两者互补。PFBA 可以用 stable-prefix 数据训练分支翻译器。

可借鉴：stable prefix 提取、prefix/continuation 拆分、置信度阈值。

局限：需要为每个前缀跑一次翻译（数据生成贵）；不做推理时未来分支。

3.4 ALTA / RALCP：放松的一致性

Wang et al., ALTA 2024. 链接

方法：LLM 增量 prompting；对 beam search 的多个候选，取"至少 α 比例候选都同意"的最长前缀：

$$\bar{y} = \mathrm{LCP}_\alpha\left(\{y^{(b)}\}_{b=1}^{B}\right)$$

而不是要求所有候选完全一致（严格 LCP）。α 控制保守程度。

结果：Llama2-7b-chat 在 MUST-C 9 种语言上 BLEU/LAAL 优于专用 MT；但作者明确指出 LLM 算力成本是落地障碍。

为什么有效：beam 候选的"多数一致"比"全体一致"更鲁棒；放松阈值可以在延迟和质量之间调。

与 TAF/PFBA 的关系：RALCP 是 TAF 多数投票的前身；PFBA 可以直接用 RALCP 作为一致提交规则。

可借鉴：RALCP、beam 一致性、α 阈值。

局限：算力成本；不预测未来。

3.5 CUNI-KIT：不改模型的 onlinization

Polák et al., IWSLT 2022. 链接

方法：用离线多语言 ST 模型，做 chunk 级 onlinization；稳定假设检测（LCP）；写比例控制；修正 AL 在长假设下的负值。

结果：延迟约为离线的 1/3；中高延迟与离线相当；低延迟有竞争力。

与 TAF/PFBA 的关系：无训练 baseline；stable prefix 的工程版。

可借鉴：onlinization、chunk size/写比例、AL 修正。

局限：稳定性判定仍要等；不预测未来。

4. 未来信息能否训练进模型

4.1 Future-Guided Incremental Transformer

arXiv:2012.12465, 2020. 链接

问题：wait-k 训练慢（隐状态重算）；训练时缺少未来源信息。

方法：

1. 增量 Transformer + average embedding layer（AEL），加速隐状态计算；
2. 用全句 teacher 做知识蒸馏，把未来信息"隐式"嵌入增量模型；
3. zh-en/de-en 实验。

结果：训练速度平均提升约 28×；质量优于 wait-k。

为什么有效：teacher 在训练时看到全句，学生被迫在只看前缀时模仿 teacher 的输出分布，从而学到"未来会怎样影响当前翻译"。

与 TAF/PFBA 的关系：训练时未来引导，不是推理时分支；可与 TAF 结合（用蒸馏模型做分支翻译器）。

可借鉴：teacher 蒸馏、增量训练。

局限：未来信息是隐式的，不可枚举/验证；不做推理时预测。

4.2 Glancing Future

arXiv:2309.06179, 2023. 链接

问题：prefix2prefix 训练丢失全局信息，导致 forced prediction。

方法：课程学习——从完整源（seq2seq）逐步缩短到前缀（prefix2prefix），让模型平滑过渡。

结果：可叠加在多种 SiMT 方法上，稳定提升。

为什么有效：模型先学会"看全句怎么翻"，再学会"只看前缀怎么翻"，减少训练-推理分布突变。

与 TAF/PFBA 的关系：训练课程；不是推理推测。

可借鉴：课程学习、未来上下文。

局限：只在训练期使用未来。

4.3 Anticipation-Free Training：反方观点

arXiv:2201.12868, 2022. 链接

问题：长距离重排序会让模型在源词未到时被迫预测目标，导致 aggressive anticipation 和幻觉。

方法：把翻译分解为**单调翻译 + 重排序**；用辅助排序网络（ASN）重排隐状态，使目标顺序匹配；推理时移除 ASN，保持流式。

结果：更低延迟下优于此前方法。

为什么重要：它指出"猜未来"不是免费的——如果训练目标强迫模型预测未读源词，模型会学到 hallucination。这为 TAF/PFBA 的风险提供了理论解释。

与 TAF/PFBA 的关系：反方证据；PFBA 必须用验证/一致提交控制 hallucination，而不是盲目 anticipation。

可借鉴：ASN、重排序、hallucination 分析。

局限：不显式枚举未来；ASN 增加训练复杂度。

4.4 NAST：非自回归流式 Transformer

arXiv:2310.14883, 2023. 链接

问题：自回归 SiMT 容易过度 anticipation。

方法：单向 encoder + 非自回归 decoder；chunk 内并行生成；用 blank/repetitive token 调整 READ/WRITE；alignment-based latency loss。

结果：优于强自回归基线。

为什么有效：NAR 在 chunk 内一次生成多个词，减少串行依赖；blank token 让模型可以"跳过"还没准备好的位置。

与 TAF/PFBA 的关系：chunk 内并行生成；不是跨未来分支。可以与 PFBA 结合：分支内用 NAR 加速。

可借鉴：intra-chunk 并行、blank token、latency loss。

局限：NAR 质量/训练难度；不展开未来。

5. LLM 机制的直接类比

5.1 Speculative Decoding：draft-verify 的数学

Leviathan, Kalman, Matias, ICML 2023. 链接

方法：draft model 提议 γ 个 token；target model 一次前向并行验证；接受/拒绝；输出分布无损。

期望接受长度（接受率 α）：

$$\mathbb{E}\left[\#\text{accepted tokens}\right] = \frac{1-\alpha^{\gamma+1}}{1-\alpha}$$

与 PFBA 的关系：PFBA 把 draft 从"目标 token"换成"源未来"；验证从"target 概率"换成"真实到达的源文"；接受/拒绝的数学可以直接借用。

可借鉴：接受/拒绝规则、期望加速、draft-verify 流程。

局限：输出侧；需要 draft model。

5.2 Self-Consistency：多采样 + 投票

Wang et al., arXiv:2203.11171, 2022. 链接

方法：采样多条推理路径；对最终答案多数投票。

与 TAF 的关系：TAF 的多数投票就是"源未来版"的 self-consistency；区别是 TAF 的"路径"是不同未来，最终要提交的是**目标前缀**，不是答案。

可借鉴：多数投票、多样性采样。

局限：同一输入；没有真实未来可验证。

5.3 Mask-Predict：并行填充

Ghazvininejad et al., EMNLP 2019. 链接

方法：训练时随机 mask 目标；推理时迭代并行填充，每轮选最自信的位置。

与 PFBA 的关系：未提交目标可以保持 mask，每来一个 chunk 并行填一批；与 `<PEND>` 方案直接相关。

可借鉴：并行填充、mask schedule、置信度选择。

局限：NAR 质量；不是流式策略。

5.4 FLARE：前瞻式主动检索

Jiang et al., EMNLP 2023. 链接

方法：先按当前上下文生成 draft；若出现低置信度 token，就用它作为 query 检索；迭代。

与 PFBA 的关系：结构最像——先向前看/先起草，再根据不确定性获取信息。区别是 FLARE 获取外部知识，PFBA 获取"真实到达的源文"。

可借鉴：置信度触发的前瞻、draft-then-verify。

局限：不是实时翻译；没有分支并行预算。

5.5 Quiet-STaR：训练模型"先想再说"

Zelikman et al., arXiv:2403.09629, 2024. 链接

方法：在每个 token 位置生成 rationale（思考），用 REINFORCE 训练；所有位置的思考可并行生成。

与 PFBA 的关系：训练模型做 anticipation 的直接类比；PFBA 是"翻译前先想未来源"，Quiet-STaR 是"说话前先想理由"。

可借鉴：辅助未来思考、并行 rationale。

局限：推理任务；计算成本高。

5.6 Simul-LLM / TransLLaMa：LLM 同传的两条基线

Simul-LLM（Agostinelli et al., ACL 2024）链接：wait-k prompt 微调；wait-7 训练比 wait-3 高最多 2.6 BLEU；开源框架。

TransLLaMa（Koshkin et al., EMNLP Findings 2024）链接：因果对齐数据 + `<WAIT>` token；LLM 自己决定读写；en-de/en-ru；GPT-4 zero-shot 也可。

与 PFBA 的关系：一个是 wait-k prompt 基线，一个是控制 token 基线；PFBA 可以套在它们之上（用它们做分支翻译器）。

可借鉴：prompt 结构、WAIT token、因果对齐。

局限：不预测未来；WAIT 有额外 decode 开销。

6. 横向对比表

文献	未来来源	提交规则	验证	并行单元	计算分析	主要结果	与 TAF 的关系
TAF	LLM 采样 n 个	多数投票 + τ	无	n 分支	有单步墙钟，无预算调度	3 词 +5 BLEU	本体
Speculative Beam Search	目标侧 hallucination	beam 排序	无	单路	无	多语言大幅提升	目标侧 lookahead
SSBD	上一轮译文	验证 + 首分歧	有	单路	1.3–1.7×	COMET 相当，闪烁降	目标侧 draft
STACL	无	wait-k	无	单路	无	低延迟可接受	"等"的基线
Liu 2020	无	LCP	稳定性	单路	有延迟/重算	83% 延迟降	一致提交祖先
Hoang	无	stable prefix	置信度	单路	数据生成贵	Pareto 改善	训练侧互补
RALCP	beam	放松 LCP	一致性	beam	算力成本	LLM 优于专用 MT	提交规则
Future-Guided	全句 teacher	蒸馏	无	单路	28× 训练加速	优于 wait-k	训练侧
Glancing Future	全句课程	训练课程	无	单路	无	多方法提升	训练侧
Anticipation-Free	无	单调 + 重排	ASN	单路	无	低延迟更好	反方/风险
NAST	无	blank token	对齐 loss	chunk 内	无	优于 AR	并行生成

文献	未来来源	提交规则	验证	并行单元	计算分析	主要结果	与 TAF 的关系
Speculative Decoding	目标 token	接受/拒绝	有	draft	有期望加速	2–3×	验证数学
Self-Consistency	目标样本	投票	无	n 样本	无	推理提升	投票机制
Mask-Predict	目标 mask	迭代填充	置信度	并行位置	无	NAR 提速	并行填充
FLARE	检索	置信度	有	单路	无	长文提升	前瞻结构
Quiet-STaR	未来 token	rationale	无	位置并行	计算贵	推理提升	anticipation 训练

7. 从文献到 PFBA 的 8 条结论

1. **TAF 是最近的先验工作，不能声称"首次提出未来分支"**；但 TAF 是算法/质量论文，不是并行系统论文。
2. **TAF 的最大缺口是算力与实时性**：0.2B MT 上单步 72→1067 ms；n 个分支没有被当作并行资源来调度。
3. **TAF 的多数投票不是验证**：28% 的实例会因 LLM 偏见引入幻觉；需要"真实未来到达后验证 + 回退"。
4. **提交规则是算法杠杆**：LCP/RALCP/stable prefix 都比裸多数投票更可控；τ 应随不确定性自适应。
5. **分支并行是系统杠杆**：共享前缀 KV、批量解码、tree attention、$B_{\max}$；把 n 从"串行开销"变成"并行预算"。
6. **训练侧可以用 stable prefix / FIM / 蒸馏**：让分支翻译器更擅长"前缀 + 想象未来"的输入。
7. **语音场景用 ASR n-best/lattice 做未来分支**：比 LLM 采样更便宜、更贴近真实不确定性；Taf 的文本结论可以迁移到级联系统。
8. **测量要包括覆盖率、接受率、oracle gap、墙钟、GPU 利用率**；如果覆盖率低，就报告"信息延迟不可预测"的负结果，而不是硬说方法有效。

8. 复现与实验清单

8.1 复现 TAF

- 模型：Llama 2 7B base + Transformer-Big；
- 数据：WMT15 De-En；WMT20 En-Zh（可选）；
- 参数：$n = 10$，$l = 10$，top-$k = 10$，$\tau \in [0.5, 1.0]$；
- 指标：BLEU、COMET、LAAL、单步墙钟；
- 对照：RALCP vs RALCP+TAF；
- 记录：覆盖率（真实未来是否在 n 个分支中）、接受率、单步耗时。

8.2 加 PFBA 的四个改动

1. **共享前缀 KV**：n 个分支只 prefill 一次源前缀；
2. **分支批处理**：n 个分支一次 decode；
3. **真实未来验证**：到达后匹配分支，保留/回退；
4. **预算调度**：测 $\rho(B) = Bc_{\text{branch}}/\Delta$，找 $B_{\max}$。

8.3 消融

- $n \in \{2, 4, 8, 16\}$；
- $l \in \{2, 4, 6, 10\}$；
- $\tau \in \{0.5, 0.6, 0.7, 0.8, 0.9\}$；
- 验证 on/off；
- 共享前缀 KV on/off；
- LLM 预测器：Llama2-7B vs MicroLlama-300M vs n-gram；
- 语言对：De-En（SOV→SVO）vs En-Zh（SVO→SVO）。

8.4 预期结论

- TAF 复现：3 词延迟下 +5 BLEU 量级；
- 共享前缀 + 批处理：$n = 10$ 的墙钟从"串行 10×"降到"接近 2–4×"；
- 验证/回退：flicker/幻觉下降，质量不降；
- $B_{\max}$：在 $\Delta = 1$ s 时，7B 模型可能只能跑 2–4 个分支；小模型可跑更多；
- 覆盖率：短 l 高、长 l 低；De-En 的动词/从句是主要收益来源。

9. 一句话总结

TAF 证明了"用 LLM 猜未来源、翻译多个分支、投票提交"能显著改善低延迟同传；它留下的是"算力太贵、没有验证、没有实时调度、没有并行系统分析"的空位。PFBA 的价值不在重复这个想法，而在把这条主线做成一个可测量、可调度、可验证的并行系统。